



최소/최대 힙, 이진 탐색

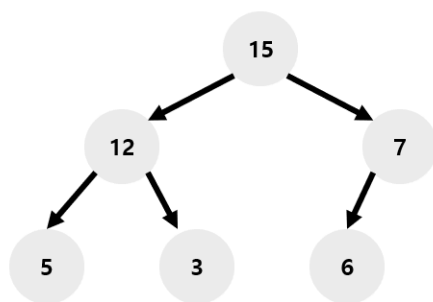
https://youtu.be/5MvvRUzzZ_o?feature=shared

<https://youtu.be/vRyVom8zdBs?feature=shared>

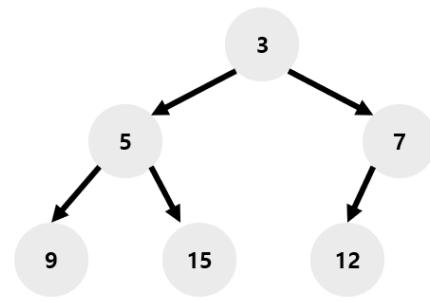
<https://youtu.be/9GLQK6SdBO8?feature=shared>

힙(Heap) 자료구조 수업노트

힙(heap)이란?



최대 힙(max heap)



최소 힙(min heap)

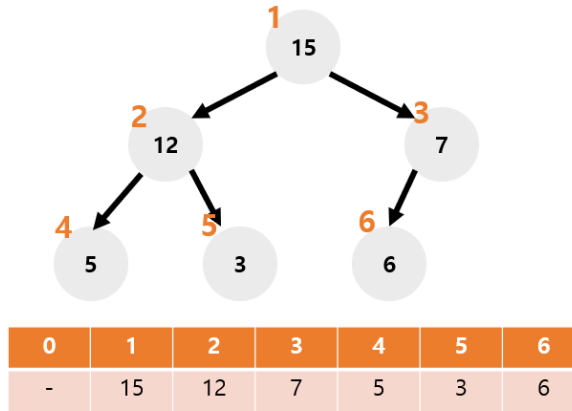
힙은 완전 이진 트리의 일종인 자료구조

- 완전 이진 트리: 마지막 레벨을 제외한 모든 레벨의 자식들이 왼쪽부터 채워진 이진 트리
- 우선순위 큐를 위하여 만들어진 자료구조이며 개의 값들 중에서 최댓값/최솟값을 빠르게 찾아내도록 만들어짐

힙의 종류

- **최대 힙(max heap):** 부모 노드의 키 값 $\geq$ 자식 노드의 키 값을 만족하는 완전 이진 트리
- **최소 힙(min heap):** 부모 노드의 키 값 $\leq$ 자식 노드의 키 값을 만족하는 완전 이진 트리
- 느슨한 정렬 상태(반정렬 상태) 유지하지만 트리의 깊이 중복 값 허용

배열로 힙 구현하기



배열로 힙 구현

부모 노드	i
왼쪽 자식 노드	2i
오른쪽 자식 노드	2i + 1

배열로 힙 구현하기는 완전 이진 트리가기 때문에 빈 공간이 없어 배열로 구현하기에 용이하다. 구현을 쉽게 하기 위해 배열의 인덱스는 1부터 사용하고, 주는 노드를 1번으로 저장하고 같은 레벨에서 좌에서 우로 저장하는 순서로 저장한다.

배열 힙 구현 인덱스 규칙

배열로 힙 구현

부모 노드 i
 왼쪽 자식 노드 2i
 오른쪽 자식 노드 2i + 1

배열 인덱스: [0, 1, 2, 3, 4, 5, 6] 힙 값: [-, 15, 12, 7, 5, 3, 6]

힙 제거과정(heapify)

힙 제거과정(heapify)함에서 원소의 삽입이나 삭제가 일어나면 최대 힙의 조건이 깨질 수 있다. 이 경우 다시 최대 힙의 조건을 만족하도록 노드의 위치를 바꾸는 것을 heapify라고 한다.

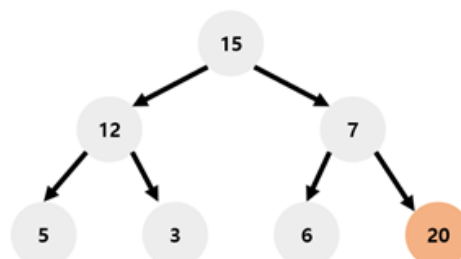
삽입과 삭제의 경우 연산 자체는 O(n)의 시간 복잡도로 작동하지만, heapify의 과정이 O(logN)이기 때문에 최종적으로 O(logN)의 시간 복잡도를 가지게 된다.

최대 힙 삽입 과정

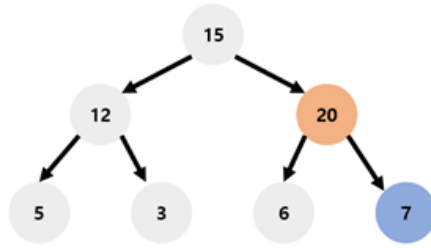
원소를 가장 맨 뒤 노드에 삽입한 다음, 가장 맨뒤 노드부터 부모 노드와 값을 비교하여 최대 힙 조건을 만족하지 않을때마다 위치를 바꿔준다.

삽입 과정 예시 (최대 20 삽입)

1. 가장 말단 노드에 원소 삽입
 (최대 힙에 원소 20 삽입)



2. 부모 노드와 비교하여 값이 크면 교환, 작으면 정지
(20이 부모 노드 7보다 크기 때문에 교환)



3. 2번 과정을 반복
(20이 부모 노드 15보다 크기 때문에 교환)



1. 가장 맨 뒤 노드에 원소 삽입 (최대 힙에 원소 20 삽입)
2. 부모 노드와 비교하여 값이 크면 교환, 작으면 정지 (20이 부모 노드 7보다 크기 때문에 교환)
3. 2번 과정을 반복 (20이 부모 노드 15보다 크기 때문에 교환)

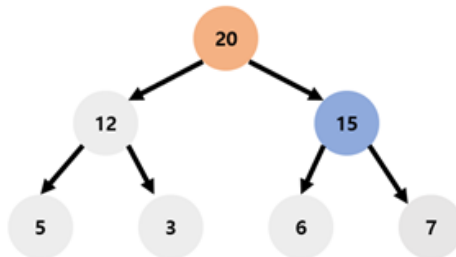
최악의 경우 가장 말단에서 삽입한 원소가 루트 노드까지 올라가게 되므로, 이때의 비교 횟수는 트리의 높이만큼이다. 따라서 삽입의 경우 heapify의 시간 복잡도는 $O(\log N)$ 이 된다.

최대 힙 삭제 과정

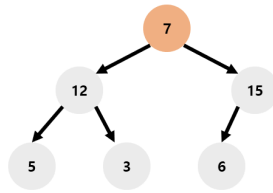
가장 원소인 루트 노드를 삭제하고 나면, 가장 맨뒤의 원소를 루트로 이동시킨다. 이후 루트 노드부터 자식 노드와 비교하여 최대 힙 조건을 만족하도록 힙을 구성한다.

삭제 과정 예시

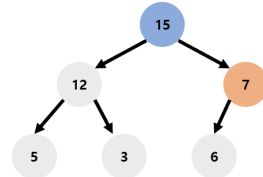
3. 2번 과정을 반복
(20이 부모 노드 15보다 크기 때문에 교환)



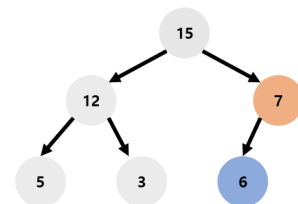
1. 루트 노드 삭제 후 가장 말단 노드를 루트 노드로 이동
(가장 말단 노드 7을 루트 노드로 이동)



2. 큰 값을 가진 자식과 위치 교환, 자신이 가장 크다면 정지
(자식 노드 12와 15 중에서 15가 더 크므로 15와 교환)



3. 2번의 과정을 반복
(자식 노드 6보다 7이 크므로 교환하지 않고 정지)



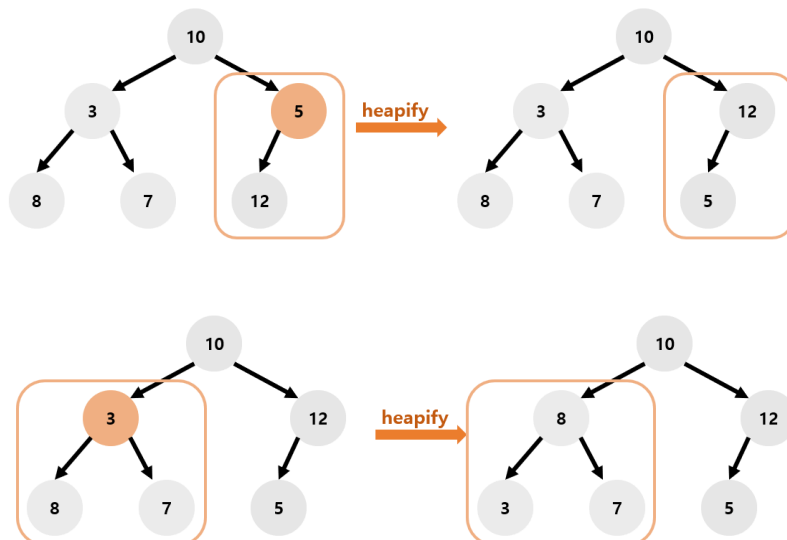
1. 루트 노드 삭제 후 가장 말단 노드를 루트 노드로 이동
2. 큰 값을 가진 자식과 위치 교환, 자식의 가장 크다면 정지
3. 2번의 과정을 반복 (자식 노드 6보다 7이 크므로 교환하지 않고 정지)

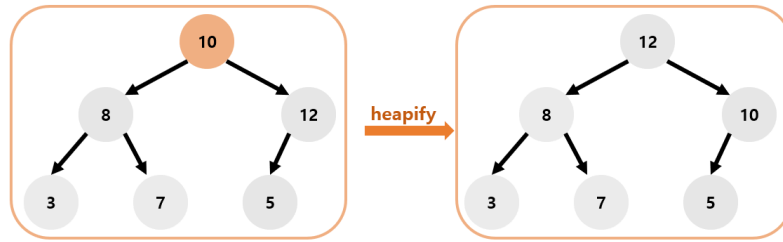
삽입 과정과 유사하게, 최악의 경우 루트 노드에서 말단 노드까지 내려오게 되므로 비교 횟수는 트리의 높이만큼이다. 따라서 삭제 과정에서 heapify의 시간 복잡도는 $O(\log N)$ 이 된다.

힙 만들기(build heap)

build heap은 임의의 배열을 힙으로 만드는 과정이다. 이때 heapify를 N번 진행하게 되므로 시간 복잡도는 $O(N \log N)$ 이다.

build heap 과정





가장 맨뒤의 오른쪽에 있는 노드의 부모 노드부터 위에서 아래로 heapify를 진행한다.

과정 예시: 10, 3, 5, 8, 7, 12 → heapify 과정을 통해 최대 힙으로 변환

최소 힙 삽입상세 예제

C++ 구현 코드

```

#include <iostream>

// 최대 힙 배열 (인덱스 1부터 사용)
int maxHeap[256] = {};
int heapSize = 0;

// 최대 힙에 원소 삽입
void push(int value)
{
    // 1. 가장 맨 뒤 노드에 원소 삽입
    heapSize++;
    maxHeap[heapSize] = value;

    int currentIndex = heapSize;

    // 2. 부모 노드와 비교하여 최대 힙 조건을 만족하도록 위치 조정
    while (currentIndex > 1) // 루트 노드가 아니라면
    {
        int parentIndex = currentIndex / 2; // 부모 노드 인덱스

        // 부모가 자식보다 크거나 같으면 최대 힙 조건 만족 -> 종료
        if (maxHeap[parentIndex] >= maxHeap[currentIndex])
            break;

        // 부모보다 크면 교환 (최대 힙 조건 위반)
        std::swap(maxHeap[parentIndex], maxHeap[currentIndex]);
        currentIndex = parentIndex; // 부모 위치로 이동하여 계속 확인
    }
}

// 최대 힙에서 최댓값(루트) 제거
int pop()
{
    // 1. 루트 노드(최댓값) 저장
    int maxValue = maxHeap[1];

    // 2. 가장 맨 뒤 노드를 루트로 이동
    maxHeap[1] = maxHeap[heapSize];
    maxHeap[heapSize] = 0;
    heapSize--;
}

```

```

int currentIndex = 1;

// 3. 루트부터 자식과 비교하여 최대 힙 조건을 만족하도록 위치 조정
while (true)
{
    int leftChildIndex = currentIndex * 2;      // 왼쪽 자식 인덱스
    int rightChildIndex = currentIndex * 2 + 1; // 오른쪽 자식 인덱스
    int largestIndex = currentIndex;

    // 왼쪽 자식이 존재하고, 현재 노드보다 크면
    if (leftChildIndex <= heapSize && maxHeap[leftChildIndex] > maxHeap[largestIndex])
    {
        largestIndex = leftChildIndex;
    }

    // 오른쪽 자식이 존재하고, 가장 큰 값보다 크면
    if (rightChildIndex <= heapSize && maxHeap[rightChildIndex] > maxHeap[largestIndex])
    {
        largestIndex = rightChildIndex;
    }

    // 현재 노드가 자식들보다 크면 최대 힙 조건 만족 -> 종료
    if (largestIndex == currentIndex)
        break;

    // 가장 큰 자식과 교환
    std::swap(maxHeap[currentIndex], maxHeap[largestIndex]);
    currentIndex = largestIndex; // 교환한 위치로 이동하여 계속 확인
}

return maxValue;
}

int main()
{
    // 최대 힙 자료구조 테스트
    // C++ STL: std::priority_queue<int> (기본값이 최대 힙)

    push(3);
    push(5);
    push(2);
    push(4);
    push(1);
    push(6);

    // 최댓값부터 차례로 출력: 6 5 4 3 2 1
    std::cout << pop() << std::endl; // 6
    std::cout << pop() << std::endl; // 5
    std::cout << pop() << std::endl; // 4
    std::cout << pop() << std::endl; // 3
    std::cout << pop() << std::endl; // 2
    std::cout << pop() << std::endl; // 1

    return 0;
}

```

힙의 특징 및 활용

시간 복잡도

- 삽입(push): $O(\log N)$
- 삭제(pop): $O(\log N)$
- 힙 생성(build heap): $O(N \log N)$

공간 복잡도

- 배열 기반: $O(N)$

주요 활용 분야

- 우선순위 큐 구현
- 힙 정렬 알고리즘
- 최단 경로 알고리즘 (다익스트라)
- 허프만 코딩 등

C++ STL

```
#include <queue>

// 최대 힙 (기본)
std::priority_queue<int> max_heap;

// 최소 힙
std::priority_queue<int, std::vector<int>, std::greater<int>> min_heap;
```

핵심 포인트

1. 완전 이진 트리: 배열로 구현하기 용이
2. 느슨한 정렬: 완전 정렬이 아닌 부분 정렬 상태
3. heapify 과정: 힙 속성 유지를 위한 핵심 연산
4. $O(\log N)$ 성능: 삽입/삭제의 효율적인 시간 복잡도
5. 인덱스 규칙: 부모(i), 왼쪽 자식(2i), 오른쪽 자식(2i+1)

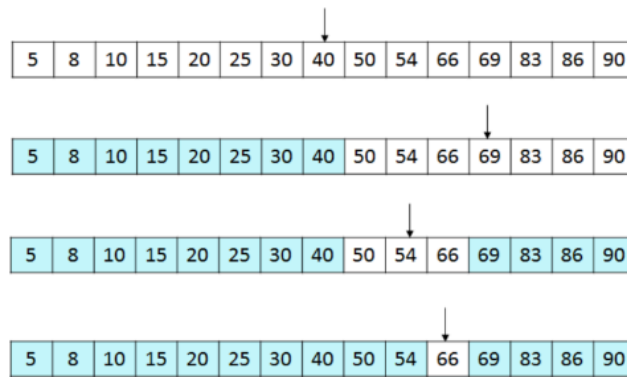
이진탐색이란?

이진 탐색이란 정렬돼 있는 데이터에서 특정한 값을 찾아내는 알고리즘이다.

- 이진탐색은 이분탐색이라고도 불린다. 이진탐색은 탐색 범위를 반으로 나누어 찾는 값을 포함하는 범위를 좁혀가는 방식으로 동작한다. 주로 배열의 인덱스를 기준으로 배열 내의 값을 탐색하는데 사용되지만, 리스트, 트리 등에서도 사용할 수 있다.
- ❌ 하지만 모든 리스트, 트리에서 사용할 수 있는 것이 아니라 반드시 원소들이 일정한 순서(예를 들면 오름차순)로 정렬된 구조에서만 사용할 수 있기 때문에 정렬된 리스트나 트리에서 사용이 가능하다.
- 이진탐색의 시간 복잡도는 $O(\log n)$ 으로, 배열의 크기에 비례하여 실행 시간이 빨라진다.
따라서 대용량 데이터에서 특정 값의 위치를 찾는 데 용이하다.

이진탐색의 동작 방식

😊 이진 탐색의 동작 방식은 우리가 생각하는 중간점 탐색소거 방식과 크게 다르지 않다.



배열에서의 동작방식

1. 정렬된 배열에서 중간 인덱스(mid)를 찾는다.
2. 찾으려는 값(target)과 중간 값(mid_val)을 비교한다.
3. target이 mid_val보다 작으면 mid를 기준으로 왼쪽 부분 배열을 탐색한다. target이 mid_val보다 크면 mid를 기준으로 오른쪽 부분 배열을 탐색한다.
4. 탐색 범위를 반으로 줄인다.
5. 탐색 범위가 더 이상 없을 때까지 위 과정을 반복한다.

예제 코드

```
#include <iostream>

#include <iostream>
#include <queue>

//binary search
int vect[10] = { 1, 2, 3, 4, 5, 6, 7, 9 };
int target = 7;

void runrecursive(int start, int end)
{
    int mid = (start + end) / 2;

    if (vect[mid] == target)
    {
        std::cout << "found" << std::endl;
        return;
    }
    else if (vect[mid] < target)
    {
        runrecursive(mid + 1, end);
    }
    else
    {
        runrecursive(start, mid - 1);
    }
}
```



```

void run()
{
    while (true)
    {
        int start = 0;
        int end = 7;
        int mid = (start + end) / 2;

        bool flag = false;
        while (start <= end)
        {
            mid = (start + end) / 2;

            if (vect[mid] == target)
            {
                flag = true;
                std::cout << "found" << std::endl;
                break;
            }
            else if (vect[mid] < target)
            {
                start = mid + 1;
            }
            else
            {
                end = mid - 1;
            }
        }

        if (flag)
        {
            break;
        }

        if (start > end)
        {
            std::cout << "not found" << std::endl;
        }
    }
}

int main()
{
    run();
    //runrecursive(0, 7);

    return 0;
}

```

“강의는 많은데, 왜 나는 아직도 코드를 못 짤까?”

혼자 공부하다 보면 누구나 이런 고민을 하게 됩니다.

- 강의는 다 들었지만 막상 손이 안 움직이고,
- 복습을 하려 해도 무엇을 다시 봐야 할지 모르겠고,

- 질문할 곳도 없고,
- 유튜브는 결국 정답을 따라 치는 것밖에 안 되는 것 같고.

문제는 '연습'이 빠졌기 때문입니다.

단순히 강의를 듣는 것만으로는 실력이 늘지 않습니다.

실제 문제를 풀고, 고민하고, 직접 구현해보는 시간이 반드시 필요합니다.

그래서, 얍얍코딩 코칭은 다릅니다.

그냥 가르치지 않습니다.

스스로 설계하고, 코딩할 수 있게 만듭니다.

얍얍코딩 코칭에서는 단순한 예제가 아닌,

스스로 문제를 분석하고 구현해야 하는 **연습문제**를 제공합니다.

이 연습문제들은 다음과 같은 역량을 키우기 위해 설계되어 있습니다:

- 문제를 스스로 쪼개고 설계하는 힘
- 다양한 조건을 만족시키는 실제 구현 능력
- 기능 단위가 아닌, 프로그램 단위로 사고하는 습관
- 마침내 자신의 힘으로 코드를 끝까지 작성하는 경험

지금 필요한 건 더 많은 강의가 아닙니다.

코드를 스스로 완성해 나가는 훈련,

그것이 지금 실력을 끌어올릴 가장 현실적인 방법입니다.

👉 자세한 안내 보기: [프리미엄 코칭 안내 바로가기](#)

👉 또는 카카오톡 상담방: [얍얍코딩 상담방](#)

"강의는 많은데, 왜 나는 아직도 코드를 못 짤까?"

혼자 공부하다 보면 누구나 이런 고민을 하게 됩니다.

- 강의는 다 들었지만 막상 손이 안 움직이고,
- 복습을 하려 해도 무엇을 다시 봐야 할지 모르겠고,
- 질문할 곳도 없고,
- 유튜브는 결국 정답을 따라 치는 것밖에 안 되는 것 같고.

문제는 '연습'이 빠졌기 때문입니다.

단순히 강의를 듣는 것만으로는 실력이 늘지 않습니다.

실제 문제를 풀고, 고민하고, 직접 구현해보는 시간이 반드시 필요합니다.

그래서, 얍얍코딩 코칭은 다릅니다.

그냥 가르치지 않습니다.

스스로 설계하고, 코딩할 수 있게 만듭니다.

얍얍코딩 코칭에서는 단순한 예제가 아닌,

스스로 문제를 분석하고 구현해야 하는 **연습문제**를 제공합니다.

이 연습문제들은 다음과 같은 역량을 키우기 위해 설계되어 있습니다:

- 문제를 스스로 쪼개고 설계하는 힘
- 다양한 조건을 만족시키는 실제 구현 능력
- 기능 단위가 아닌, 프로그램 단위로 사고하는 습관
- 마침내 자신의 힘으로 코드를 끝까지 작성하는 경험

지금 필요한 건 더 많은 강의가 아닙니다.

코드를 스스로 완성해 나가는 훈련,

그것이 지금 실력을 끌어올릴 가장 현실적인 방법입니다.

👉 자세한 안내 보기: [프리미엄 코칭 안내 바로가기](#)

👉 또는 카카오톡 상담방: [얏얏코딩 상담방](#)

▼ oopy:hide

연습문제



문제 1번

정렬 되어있는 데이터를 검색할 때, for문으로 찾는 것보다 Binary Search로 더 빨리 원하는 값을 찾을 수 있습니다.
(주의: Binary Search와 Binary Search Tree를 헷갈리지마세요!)



위 배열을 하드코딩 해 주세요.

숫자 하나를 입력받고, 배열 안에 존재하는 숫자인지 아닌지 찾는 프로그램을 작성 해 주세요.

존재한다면 O, 존재하지 않으면 X를 출력하면 됩니다.

(재귀호출을 이용해서 코딩을 해 주세요.)

입력 예제

20

출력 결과

O



문제 2번

북카페를 개업한 사장님은, 보유한 책 들을 미리 사전순으로 정렬 해 두고 Binary Search로 손님이 원하는 책을 빠르게 찾아 대여 해 주려고 합니다.



N개의 책 이름을 입력 받고, 아래 그림처럼 사전순 오름차순으로 정렬 해주세요.

(대문자 < 소문자)

사장님은 하나의 책 제목이 손님이 원하는 책 제목과 비교하는데 1 초가 걸립니다.

M명의 단골 고객이 원하는 책 제목과, 고객님이 준 시간(S) 안에 책을 찾아야 합니다.



단골 고객님이 준 시간 안에 책을 찾아줄 수 있으면 "pass",

책이 없거나, 손님이 준 시간안에 책을 찾아줄 수 없다면 "fail" 를 출력하세요.

1. ex) 단골 고객이 사장님께 2 초 시간을 준다면, 사장님은 책을 2 번까지 탐색할 수 있습니다.

[입력]

책의 수 N와 N개의 책 제목들을 입력 받습니다. ($1 \leq N \leq 100,000$)

그리고 단골 고객의 수 M 을 입력 받습니다. ($1 \leq M \leq 1,000$)

그 다음줄 부터 M명의 고객들이 찾는 책 제목, 허용되는 검색 시간 S 를 입력 받아 주세요.

[출력]

M명의 고객들에 대해서

허용된 시간 안에 책을 찾아줄 수 있는지 pass 또는 fail을 각각 출력하면 됩니다.

입력 예제

6

Rabbit Moon Opening Alien Power Ai

3

Opening 5

Alien 2

Ai 2

오름차순 정렬 결과

Ai Alien Moon Opening Power Rabbit

출력 결과

pass

pass

fail



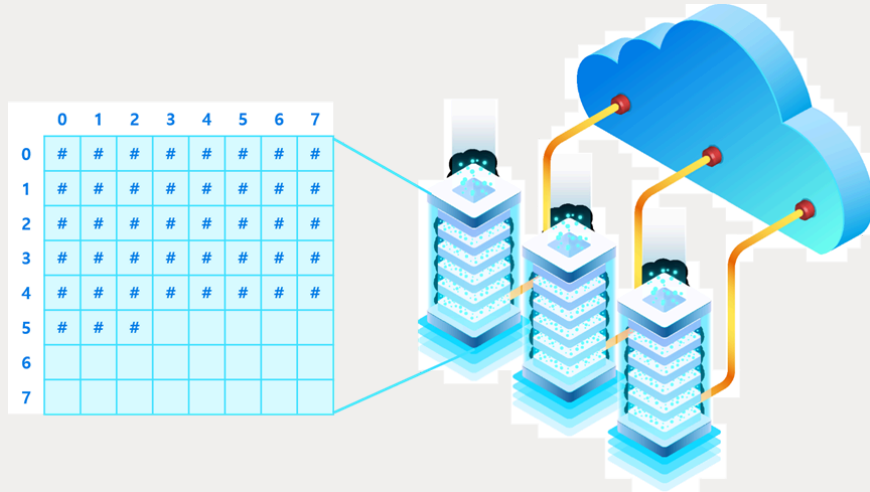
문제 3번

저장장치에 있는 데이터를 클라우드에 백업하려고 합니다.

데이터의 범위를 알아야 전송이 가능하기 때문에, 저장장치내에 어디까지 데이터가 적혀있는지 알아내야 합니다.

아래 그림처럼 $N \times N$ 의 데이터 저장장치가 있습니다.

데이터는 #으로 표시되며, (0, 0)에서부터 차례로 쌓입니다.



데이터 백업을 위해, 저장장치내에 데이터가 어디까지 쓰여있는지 확인해주세요.

$O(\log N)$ 의 속도로 데이터를 검색해야 하며, 마지막 # 데이터의 좌표를 출력하면 됩니다.

[입력]

저장장치의 가로, 세로 N 이 입력 됩니다. ($3 \leq N \leq 1,000$)

그리고 데이터 #가 적혀 있는 저장장치 내의 정보가 입력 됩니다.

[출력]

마지막 데이터가 적혀 있는 위치를 $y \times x$ 좌표로 출력 해주세요.

입력 예제

8

```

#####
#####
#####
#####
#####
###00000
00000000
00000000

```

출력 결과

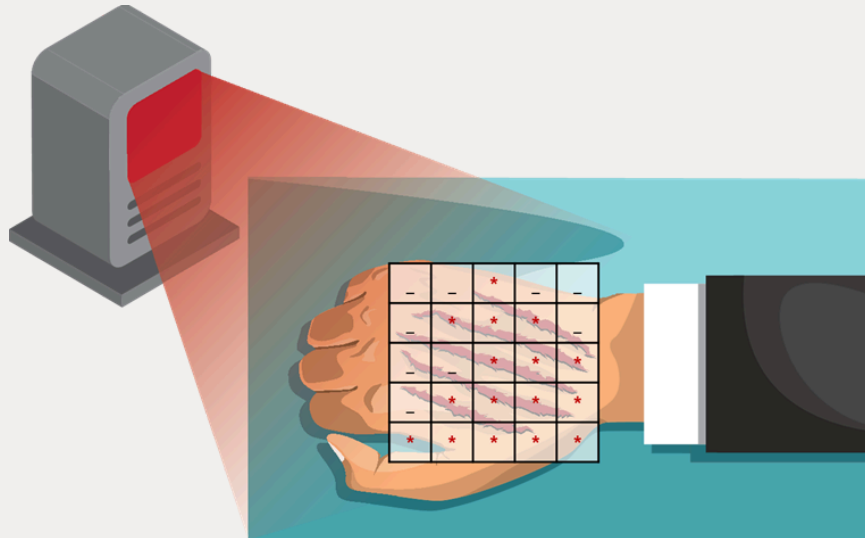
5 2



문제 4번

약국에는 새로운 기계를 들여 뒀습니다.

이 기계는 반창고 스캐너라고 불리는 기계로, 환자가 자신의 상처를 스캔하면 자동으로 상처 크기만큼 반창고를 잘라 공급해주는 스캐너 자판기입니다.



위 그림처럼 스캐너에 상처를 스캔하면, 상처의 크기를 측정합니다.

빠른 속도로 각 행마다 상처의 길이를 측정하여 반창고를 잘라 제공합니다.

많은 환자를 받기 위해 $O(\log N)$ 으로 연산을 처리하여 상처 길이를 측정해야 합니다.

(상처는 항상 mid 중간값이 포함 됩니다.)

[입력]

총 다섯 줄의 상태가 입력됩니다.

($3 \leq$ 한 문자열의 길이 $\leq 1,000$)

상처가 아닌 부분은 "_" 로 표시되고, 상처는 "*" 로 입력 됩니다.

[출력]

각 행마다 상처의 길이를 출력해주세요.

입력 예제

```

"   _   *   _   " ,
"   _   ***   _   " ,
"   _   ****   _   " ,
"   _   ****   _   " ,
"   _   ****   _   "

```

출력 결과

```

1
3
10
4
6

```



문제 5번

게임 프로그래머인 최 프로님은 3D 좌표를 계산할 때, 루트 값을 구해야하는 경우가 많습니다.
따라서 루트를 빠르게 계산하는 프로그램을 만들기로 했습니다.



입력 받은 값을 Binary Search 를 이용하여 루트 값의 정수부를 구해주세요.

만약 16 입력시 출력결과는 4 입니다.

만약 17 입력시 root 값은 4.xxx 이므로, 출력결과는 4 입니다.

만약 25 입력시 root 값은 5 입니다.

만약 35 입력시 root 값은 5.xxx 이므로, 출력결과는 5 입니다.

힌트

“루트를 계산”하는 게 아니라, 정확히는

(루트의 정수 부분) = $x * x \leq N$ 을 만족하는 가장 큰 정수 x

x 를 0부터 올리면서 $x * x \leq N$ 을 검사한다고 해보자.

$N=17$ 일 때:

x	x^2	$x^2 \leq 17$?
0	0	✓
1	1	✓
2	4	✓
3	9	✓
4	16	✓
5	25	✗
6	36	✗
7	49	✗

우리는 x 의 범위를 잡고 (0 ~ 17)

중간값 mid 를 찍어서 “너무 크냐 / 아직 작냐”를 판단해.

mid 를 사용하는 이유는 값을 반으로 나누는게 아니라

답이 있을 “구간”을 반으로 줄이는 알고리즘

목표점

- 조건: $mid * mid \leq 17$ 이면 “아직 작거나 적당함(✓)”
→ 더 큰 값도 가능하니 오른쪽으로
- 반대로 $mid * mid > 17$ 이면 “너무 큼(✗)”
→ 왼쪽으로

진행 표 (N=17)

step	left	right	mid	mid^2	판정	다음
1	0	17	8	64	✗	right = 7
2	0	7	3	9	✓	answer=3, left = 4
3	4	7	5	25	✗	right = 4

step	left	right	mid	mid ²	판정	다음
4	4	4	4	16	✓	answer=4, left = 5

[입력]

숫자 값을 하나 입력 받습니다.

[출력]

입력 받은 값의 루트 값을 정수 부분만 출력 해주세요.

입력 예제

17

출력 결과

4



문제 6번



주유를 할 때, 기름이 어느정도 찼는지 알려주는 프로그램을 작성하려고 합니다.
for문으로 짜기에는 인식속도가 너무 느리기에 Binary Search를 이용하려고 합니다.



기름의 상태를 문자열로 입력받아주세요. (총 10글자)

Binary Search를 이용하여 10칸 중 총 몇 % 기름이 채워졌는지 출력 해 주세요.

(최소 한 칸은 항상 차 있습니다)

입력 예제

#####

출력 결과

60%



문제 7번

「Heap은 Data를 이진트리 형태로 저장해두고, 우선순위가 높은 값을 빠르게 찾아주는 자료구조 입니다. 」

띄어쓰기를 제외하고 대소문자, 특수문자가 섞인 한 문장을 입력받아주세요.

Max Heap에 문자열을 구성하는 알파벳들을 넣어두고, 아스키코드 순서대로 내림 차순으로 정렬하여 출력 해 주세요.
(최대 50글자)

[STL : Max Heap]

```
#include <queue>
priority_queue<int> t;
//priority_queue<int, vector<int>, less<int>>> t;
을 간략하게 표현한 것이다.
C++
```

[STL : Min Heap]

```
#include <queue>

priority_queue<int, vector<int>, greater<int>>> t;
```

[STL : 다중 조건 우선순위 큐]

```
#include <queue>

int main()
{
    //std::priority_queue<int> default 값은 내림차순
    //std::priority_queue<int, std::vector<int>, std::less<int>>> t;
    std::priority_queue<int, std::vector<int>, std::greater<int>>> pq;

    pq.push(10);
    pq.push(30);
    pq.push(20);
    pq.push(10);
    pq.push(50);
    pq.push(15);

    return 0;
}
```

입력 예제

DoYouWannaBuildASnowMan?

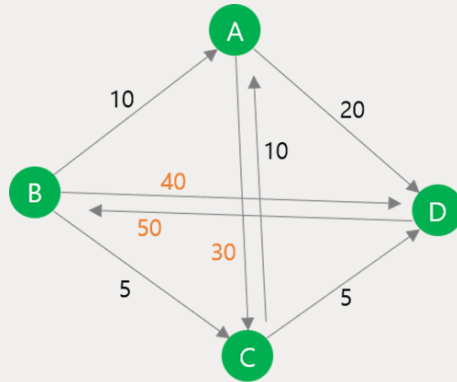
출력 결과

wuuooonnnnlidaaaYWSMDBA?

💡 문제 8번



n개 도시의 여행경로 그래프(인접행렬)를 입력받아주세요.
이 중에 값 비싼 항로 Top 3개만 판매를 하려고 합니다.



	A	B	C	D
A			30	20
B	10		5	40
C	10			5
D		50		

간선의 길이와 정보들을 모두 Heap에 넣어두고 (구조체 Heap)
가장 비싼 간선 3개를 뽑아 출력 해 주세요.

입력 예제

```
4
0 0 30 20
10 0 5 40
10 0 0 5
0 50 0 0
```

출력 결과

```
D-B 50
B-D 40
A-C 30
```

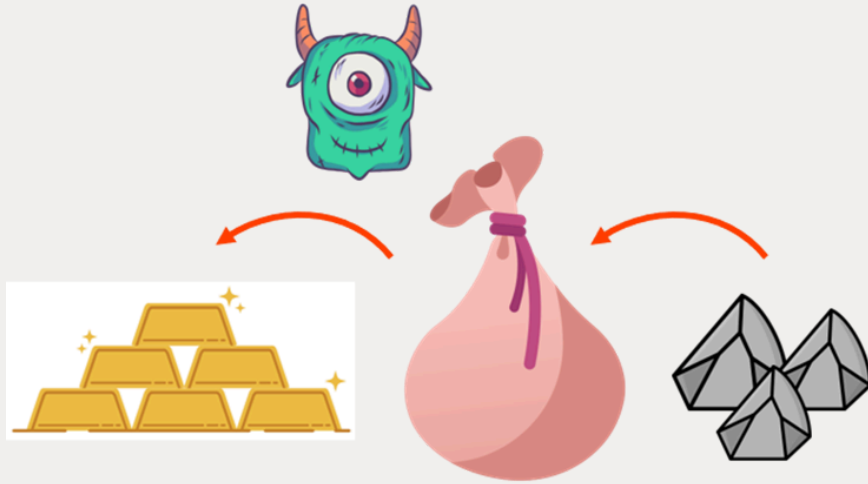


문제 9번

도깨비의 보자기에는 n 개 황금이 들어있습니다.

이 보자기는 한번 꺼낼 때 가장 가벼운 물건만 꺼내지는 마법의 보자기입니다.

보자기에 있는 황금을 꺼내면 보자기의 무게는 점점 가벼워집니다.



도깨비에게 들키지 않도록, 황금을 2개씩 꺼낼 때 마다 무거운 짱돌 1개를 넣어두려고 합니다.

황금의 개수 n 과

보자기에 들어있는 황금의 무게들을 입력 받습니다.

보자기에서 2 개의 내용물을 꺼냅니다.

그리고 마지막으로 꺼낸 황금의 2 배 무게를 가진 짱돌 1개 넣습니다.

꺼내어진 돌이 황금이 아닐 때까지 위 동작을 반복 합니다.

도깨비보자기에서 꺼내어진 황금의 개수를 출력 해 주세요.

[예시]

보자기에 1 3 3 4 9 무게의 황금이 보자기에 들어 있습니다.

1회 시도 : 보자기에서 1, 3 무게의 황금을 뺀 후, $3 \times 2 = <6>$ 무게의 짱돌을 넣는다. -----> 3, 4, $<6>$, 9

2회 시도 : 보자기에서 3, 4 무게의 황금을 뺀 후, $4 \times 2 = <8>$ 무게의 짱돌을 넣는다. -----> $<6>$, $<8>$, 9

3회 시도 : 보자기에서 꺼낸 돌이 짱돌 $<6>$ 이기에 동작을 그만 둔다.

총 4개의 황금을 얻을 수 있기에, 출력결과는 4 입니다.

[세부사항]

1. 황금과 짱돌이 같은 무게일때는, 황금이 먼저 나와야 합니다.
2. 꺼낸 돌이 짱돌일 경우, 즉시 꺼내는 작업을 중단합니다.

입력 예제

5

1 3 3 4 9

출력 결과

4



문제 1번

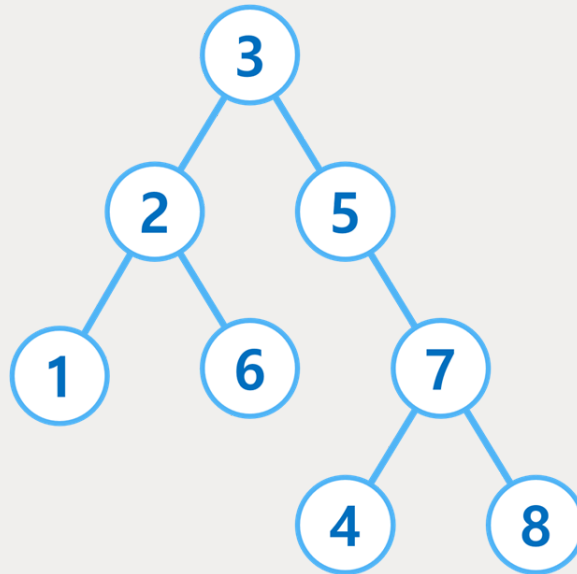
아래 이진트리를 1차원 배열에 하드코딩 해 주세요

DFS를 돌리면서 탐색 순서대로 노드의 값을 출력하고자 합니다.

출력하는 타이밍은 아래와 같습니다.

왼쪽으로 진입했다가 돌아온 뒤 (왼쪽으로 재귀),

오른쪽으로 진입했다가 돌아왔을 때 (오른쪽으로 재귀) 출력합니다.



[입력]

입력 값은 없습니다.

[출력]

출력결과는 1 6 2 4 8 7 5 3 입니다

출력 결과

1 6 2 4 8 7 5 3



문제 2번

5개국 여행 계획을 세워주는 여행사에서 경로추천을 해주려합니다.

DFS를 활용하여 시작점부터 도착지점까지의 최소 비용과 경로를 출력해주세요.

아래 그래프를 인접행렬로 하드코딩해주세요.



[입력]

시작점과 도착지점을 입력 받으세요.

[출력]

최소 비용과 경로를 출력해주세요.

갈 수 있는 길이 없다면 "impossible" 을 출력 하세요.

입력 예제

G B

출력 결과

6:GTHB



문제 3번

입력받은 숫자 갯수로 10을 만들 수 있는 조합의 수를 출력하세요.
사용 되는 숫자의 범위는 1 ~ 9 입니다.

N = 3

$$\boxed{} + \boxed{} + \boxed{} = \boxed{10}$$

N = 2

$$\boxed{} + \boxed{} = \boxed{10}$$

N = 1

$$\boxed{} = \boxed{10}$$

만일 입력 받은 N = 3 이라면, 3개의 숫자를 더하여 10이 되는 숫자 조합은 총 64개 입니다.

$$1 + 1 + 8 = 10$$

$$1 + 2 + 7 = 10$$

$$1 + 3 + 6 = 10$$

.

.

.

$$7 + 2 + 1 = 10$$

$$8 + 1 + 1 = 10$$

[입력]

1 ~ 9 의 숫자 중, 사용될 숫자의 개수 N을 입력 하세요.

[출력]

N 숫자의 개수를 조합하여 10이 되는 경우의 수의 총합을 출력하세요.

입력 예제

3

출력 결과

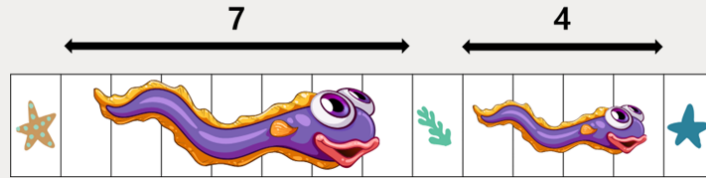
36



문제 4번

수족관에 일렬로 장어들이 지나가고 있습니다.

무료 시식권이 생겨 이 중 한마리를 시식하려고 할 때, 가장 긴 장어를 찾아 출력 해 주세요.



[입력]

최대길이 100인 문장이 입력 됩니다.

장어는 ~ 로 표시되고, 나머지 물체들은 #으로 표시 됩니다.

[출력]

가장 긴 장어를 출력 해주세요.

입력 예제

#~~~~~#~~~~#

출력 결과

~~~~~



#### 문제 5번

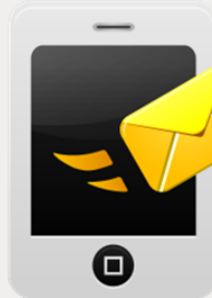
문자 메시지에 다이어트에 도움이 안 되는 단어가 있으면, 모두 3개의 #으로 (###) 모자이크 처리하려고 합니다.

성공적인 다이어트를 위해, 문자메세지 필터기능을 완성시켜주세요.

모자이크 처리 할 5개의 위험 단어들은 다음과 같습니다.

대소문자 구분 없이 위험 단어가 있으면 모자이크 처리 해 주세요.

|         |
|---------|
| chicken |
| pizza   |
| jockbal |
| bread   |
| samhap  |



#### [입력]

문장의 길이가 최대 50 글자인 한 문장을 입력 받습니다.

#### [출력]

모자이크해야할 단어를 길이에 상관없이 "###" 로 바꿔 문장을 출력해주세요.

#### 입력 예제

HeyBread!DoYouWannaBuildAChicken

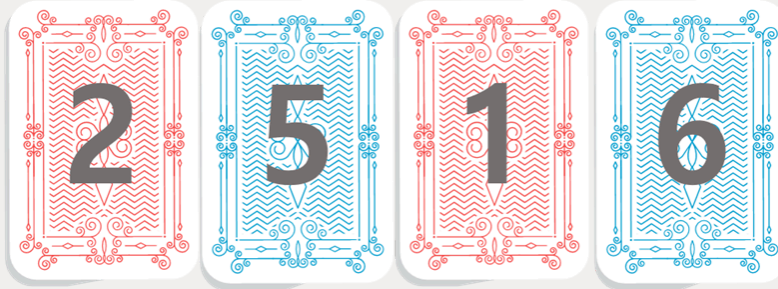
#### 출력 결과

Hey###!DoYouWannaBuildA###



### 문제 6번

숫자카드 4개를 입력 받고 숫자 카드를 재귀호출을 이용하여 조합해주세요.  
조합한 숫자가 3000이 넘는 경우가 총 몇개인지 출력 해주세요.



#### [입력]

숫자 4개를 입력 하세요.

#### [출력]

경우의 수의 총합을 출력하세요.

#### 입력 예제

2 5 1 6

#### 출력 결과

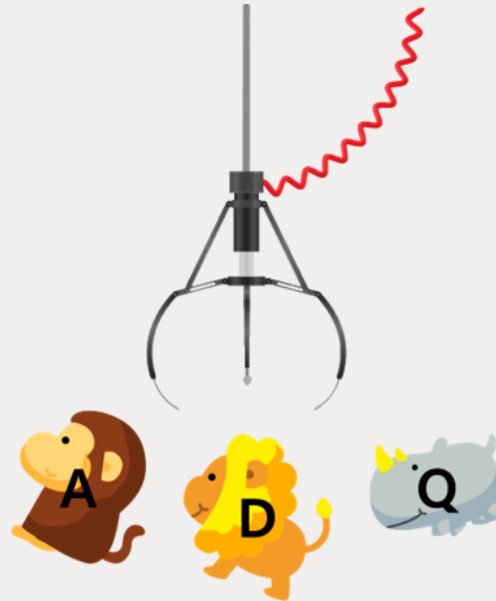
12





### 문제 7번

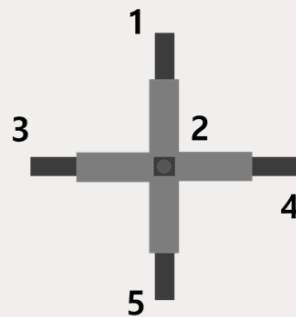
2차원 배열 인형뽑기 기계에는 인형들이 불규칙하게 배치되어 있습니다.  
그리고 인형에는 각각의 알파벳이 부여되어 있습니다.



인형 뽑기 집게의 좌표와 모양을 입력 받아 인형을 뽑았을때의 문자를 출력해주세요.

#### [Type 1 집게 모양]

위, 가운데, 왼쪽, 오른쪽, 아래 순서대로 인형의 알파벳이 출력됩니다.

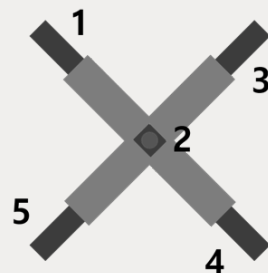


|   |   |   |   |   |
|---|---|---|---|---|
| A | E | K | X | K |
| Z | O | Q | K | P |
| C | F | W | L | X |
| C | E | I | H | B |
| N | E | M | I | O |

정답 : EOZQF

#### [Type 2 집게 모양]

왼쪽 위, 가운데, 오른쪽 위, 오른쪽 아래, 왼쪽 아래 순서대로 인형의 알파벳이 출력됩니다.



|   |   |   |   |   |
|---|---|---|---|---|
| A | E | K | X | K |
| Z | O | Q | K | P |
| C | F | W | L | X |
| C | E | I | H | B |
| N | E | M | I | O |

정답 : CEWMN

#### [입력]

인형뽑기 기계의 가로, 세로 N이 입력 됩니다. ( $3 \leq N \leq 1,000$ )

그리고 집게 명령 개수  $K$  가 입력됩니다.

인형이 들어있는 인형 뽑기 기계 내의 정보가 입력 됩니다.

그 다음 줄 부터  $K$  개수의 집게 좌표  $Y\ X$  와 집게 형태  $M$  를 입력 받아 주세요.

**[출력]**

입력 받은 집게 내림 명령 만큼의 문장을 출력 해주세요.

### 입력 예제

```
5 2
A E K X K
Z O Q K P
C F W L X
C E I H B
N E M I O
1 1 1
3 1 2
```

### 출력 결과

```
EOZQF
CEWMN
```



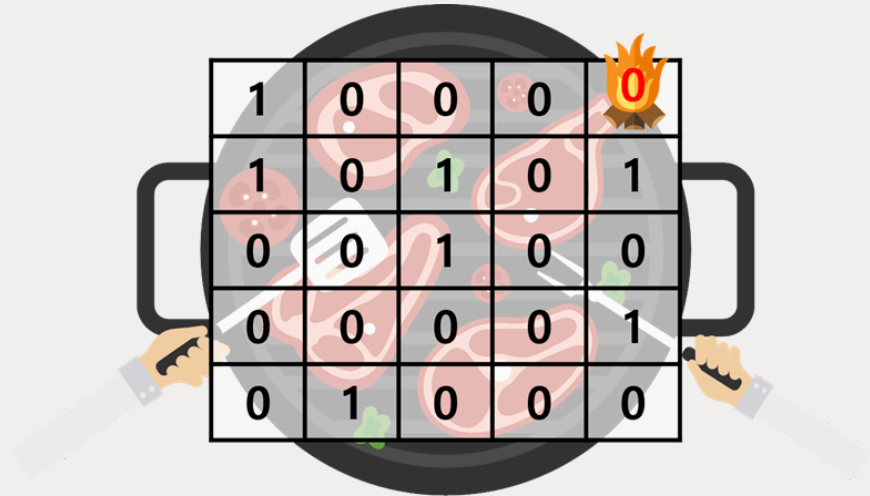
### 문제 8번

2차원 배열의 그릴에 불을 놓을 수 있는 공간은 0, 불을 놓을 수 없는 공간은 1로 입력됩니다.

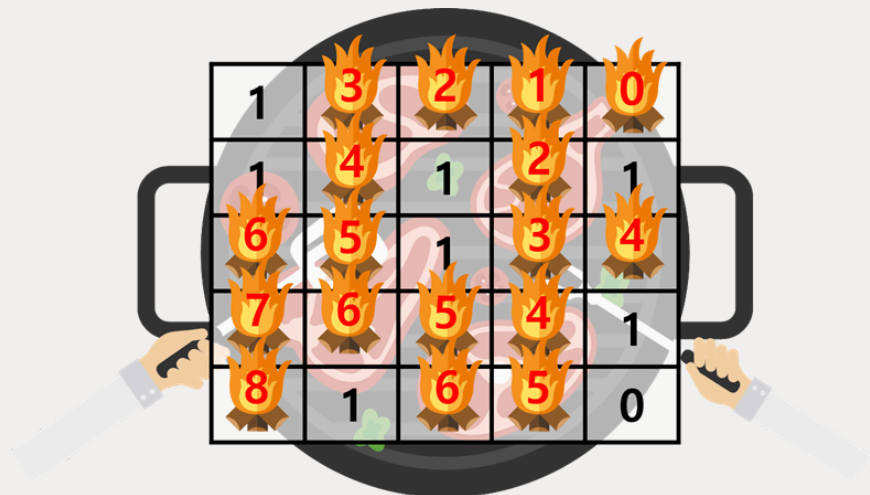
고기를 맛있게 굽기위해 불을 처음 놓을 좌표를 입력 받으세요.

불이 상, 하, 좌, 우로 한 칸씩 늘어날때마다 1초가 경과합니다.

불이 모든 자리에 붙었을때의 시간을 출력하세요.



2차원 배열에서 (0, 4)에 불을 놓는다면 2차원 배열안의 모든 0에 불이 퍼질때 까지 걸리는 시간은 8초입니다.



#### [입력]

그릴의 세로, 가로 길이 N 을 입력 받습니다. ( $3 \leq n < 1,000$ )

그릴의 정보가 입력 됩니다.

그리고 불이 처음 놓일 좌표 Y X 가 입력됩니다.

#### [출력]

모든 '0' 자리에 불이 번지는데 걸리는 시간을 출력하세요.

#### 입력 예제

```
5 5
1 0 0 0 0
1 0 1 0 1
0 0 1 0 0
0 0 0 0 1
0 1 0 0 0
0 4
```

#### 출력 결과





### 문제 9번

수압이 좋지 않은 집을 수리하기 위해, 배관을 선택해 주세요.



|    |    |    |    |
|----|----|----|----|
| 3  | 4  | -2 | 6  |
| 0  | -4 | 4  | 1  |
| -4 | 4  | 0  | -2 |
| 5  | 1  | 6  | 9  |

전체 배관의 맵은 2차원 배열로 주어지고 각 행에서 0이 아닌 칸을 한 개씩 열어야 합니다. 총 수압은 각 행의 칸을 선택했을때, 선택한 칸들의 곱입니다.



|    |    |    |    |
|----|----|----|----|
| 3  | 4  | -2 | 6  |
| 0  | -4 | 4  | 1  |
| -4 | 4  | 0  | -2 |
| 5  | 1  | 6  | 9  |

위 그림처럼, 첫 번째 줄의 6, 두 번째 줄의 4, 세 번째 줄의 4, 네 번째 줄의 9를 선택한다면 총 수압의 값은 864로 수압의 값이 가장 큰 상태가 됩니다. 또한 최대 값이 중복될 수 있습니다. 모든 경우의 수에서 가장 큰 값을 출력해주세요.

#### [입력]

배관 사이즈의 세로, 가로 N 이 입력됩니다. ( $3 \leq N \leq 1,000$ )

배관 사이즈의 정보가 음수와 양수로 입력 됩니다.

#### [출력]

배관을 열어, 선택한 배관들의 곱이 가장 큰 숫자를 출력하세요.

### 입력 예제

4

3 4 -2 6

0 -4 4 1

• 4 4 0 -2

5 1 6 9

## 출력 결과

864



### 문제 10번

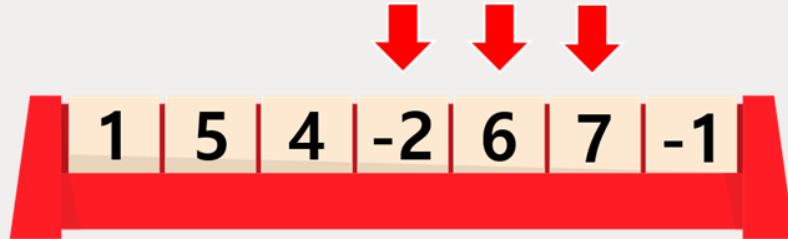
다빈치 코드게임을 벤치 마킹하여 다빈치 민코드라는 게임을 만들었습니다.

서로 M 개의 카드를 상대방의 카드패에서 뽑고, 카드들의 각각의 값들을 곱한 값으로 승패를 가립니다.

이길 확률을 높이려면 상대방이 내 카드에서 M 개를 뽑아 그 숫자들의 곱이 최소값이 되도록 유도해야 합니다.



7개의 카드패가 주어 지고 M이 3이라면 카드패들의 곱이 가장 작은 값이 나와야 합니다.



만일 M 이 3이라면 -2, 6, 7을 뽑도록 유도하여 최소값인 -84 가 나오도록 해야 이길 확률이 높아 집니다.

#### [입력]

숫자 카드의 개수 N 이 입력 됩니다. ( $3 \leq N \leq 1,000$ )

그 다음줄에, 음수와 양수의 값을 가진 카드들이 입력 됩니다.

그리고, 뽑아야할 카드 개수 M 이 입력 됩니다. ( $2 \leq M < N$ )

#### [출력]

M 개를 선택하여 곱한 값이 최소값이 될때의 값을 출력하세요.

### 입력 예제

7 3

1 5 4 -2 6 7 -1

## 출력 결과

- 84